Experiment 3: Searching and Sorting

Aim:
Write a C program to create a student database using an array of structures. Apply searching (Linear and Binary Search) and Sorting techniques (Insertion Sort, Selection sort, Shell Sort)

Objective:
- To understand use of an array of structures for maintaining records
- To implement and analyze searching techniques.
- To implement and analyze sorting techniques.
- To Compare Insertion sort, Selection sort, Shell Sort.

Theory:
Searching and sorting are two of the most fundamental operation in data structures and algorithms. They are widely used in comp-uter applications such as database management systems, search engines, operating systems, e-commerce platforms, and information retrieval systems. Efficient searching and sorting techniques improve the performance of software by reducing the time required to locate and organize data.

- Searching

Searching is the ~~type~~ process of finding the location of a specific element, called the key, within a collection of data. If the element is present, the search operation returns its position; otherwise, it indicates that the element is not found.

Searching is an essential operation in applications such as student record management, library systems, banking systems, inventory management, and online shopping.

Types of Searching:

i. Linear Search: Linear search, also known as Sequential Search, is the simplest searching technique. In this method, each element of the list is compared one by one with the target element until a match is found or the entire list has been searched.

ii. Binary Search: Binary Search is an efficient searching algorithm that works only on sorted data. It repeatedly divides the search interval into two halves until the target element is found or the search space becomes empty.

Implementations:

- Platform:

i. 64-bit Open Source Linux or its derivatives.

ii. Open source C Programming tool like gcc/Eclipse Editor.

- Test Conditions:

i. Input at least five record.

ii. Search a student in the record as per the roll number (linear search).

iii. Arrange list of students roll call list wise ~~as per roll number in the sorted list (B~~ (Use Insertion Sort, selection Sort, shell Sort).

Sundaram

FOR EDUCATIONAL USE

iv. Search a student as per roll number in the sorted list (Binary Search)
v. Test the searching and sorting methods for atleast five records.

- Pseudo Code:

```
Structure Student:
        string name
        long prn
        integer rollNo
        string mobile
        string panel

Main:
    Array of Student s[100]
    integer count=0, choice, target, res

    Print "Enter Student Database"
    While count<5 :
        Print "Enter details for Student " + (count+1) + ":"
        Print "Name, PRN, Roll no., Mobile, Panel:"
        Read  s[count].name
              s[count].prn
              s[count].rollNo
              s[count].mobile
              s[count].panel
        count++
    End while
```

```
Loop:
    Print "\n---Menu---\n"
    Print "1. Add another record"
    Print "2. Display all records"
    Print "3. Linear Search(by Roll No)
    Print "4. Binar Search (by Roll No)
    Print "5. Exit"
    Print "Enter choice:"
    Read choice

Switch(choice):
    Case 1: Print "Enter Name, PRN, Roll No, Mobile, Panel:"
            Read s[count].name
            s[count].PRN prn
            s[count].Ro'roll No
            s[count].mobile
            s[count].panel
        count++
        break
    Case 2: For i from 0 to count-1:
            Print *s[i].roll No + "|" + s[i].name + "|" +
                  s[i].prn + "|" + s[i].mobile + "|" + s[i].panel
        End for
        break
    Case 3: Print "Enter Roll no to be searched"
        Read target
        res = linearSearch (s, count, target)
        If res != -1:
            print "Found!"
```

Sundaram
FOR EDUCATIONAL USE

```
                print the respective record in tabular format
            Else:
                Print "Record Not Found"
            End If
            Break.
        Case 4: Print "Enter Roll No to search:"
            Read target
            res = binarySearch (s, count, target)
            If res != -1:
                print "Found!"
                print the respective record in tabular format
            Else
                Print "Record Not Found"
            End If
            Break
        Case 5: Exit Program
    End Loop
End Main

Function linearSearch (s, size, target):
        For i from 0 to size-1:
            If s[i].rollNo == target:
                Return i
            End If
        End For
        Return -1
End Function
```

```
Function binarySearch(s, size, target):
    integer low = 0, high = size - 1
    While low <= high:
        integer mid = (low + high) / 2
        If s[mid].rollNo == target:
            Return mid
        Else If s[mid].rollNo < target:
            low = mid + 1
        Else:
            high = mid - 1
        End If
    End while
    Return -1
End Function
```

Time complexity:

Case	Linear Search	Binary Search
Best	O(1)	O(1)
Average	O(n)	O(logn)
Worst	O(n)	O(logn)
Space Complexity	O(1)	O(1) iterative / O(logn) recursive
Data Requirement.	Unsorted or Sorted arrays	Must be sorted

Sundaram®
FOR EDUCATIONAL USE

FAQs:

1. What is the meaning of database? How to maintain it in C?

Ans. Database is an organised collection of related data stored in for easy access, retrived, updating and management.
Eg. student records containing roll no, name, marks and department.

To maintain it in C,

i. Use a structure.
ii. Use an array of structure to store multiple records
iii. Perform operations like update, delete, adel, display, search, sort, etc.
iv. Store data temporarily in memory or permanently using files.

2. What are the applications of searching

Ans. The applications are:

i. Student database management
ii. Library management
iii. Banking & ATM systems
iv. Employee record system
v. Contact list system.

3. Compare blw linear & binary search.

Feature	Linear Search	Binary Search
Data req.	Sorted / Unsorted array	Sorted array
Method	One by one	Divides space into tables
Time comp.	O(n)	O(log n)
Speed	Slower	Faster
Implement^n	Simple	Moderate
Memory	No extra	No extra
Suitable for	Small list	Large list

Sundaram FOR EDUCATIONAL USE

- Sorting

Theory:

i. Bubble Sort: It repeatedly compares adjacent elements and swaps them if they are in wrong order. After each pass, the largest element moves to it's correct position, similar to the bubbles rising to the surface.

ii. Selection Sort: It repeatedly selects the smallest element from the unsorted portion of the list and places it at the beginning.

iii. Insertion Sort: It builds a sorted list, one at a time by inserting each new element into its correct position in the already sorted position of array

iv. Shell Sort: It allows the exchange of elements that are far apart. It sorts elements by comparing and swapping items separated by a specific gap, gradually reducing the gap until it becomes 1.

v. Merge Sort: It is a divide-and conquer algorithm that divides the array into smaller subarrays, sorts each subarray recursively, and then merges them into a sorted array.

vi. Quick Sort: It selects a pivot element & ~~pra~~partitions the array into two parts such that the element smaller than the pivot are placed before it and larger elements after it.

Sundaram®

FOR EDUCATIONAL USE

Time complexity.

Case	Bubble	Selection	Insertion	Shell	Merge	Quick
Best	$O(n)$	$O(n^2)$	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Average	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^{4/3}) - O(n^{3/2})$	$O(n \log n)$	$O(n \log n)$
Worst	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n^2)$
Space C.	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(\log n)$
Stability	Stable	Unstable	Stable	Unstable	Stable	Unstable

FAQs:

1. What are applications of sorting?

Ans. The applications are:

i. Fast searching
ii. Student marks arrangement
iii. Database indexing
iv. Payroll processing
v. Data analysis.

2. Compare and contrast selection sort, insertion sort & shell sort.

Ans.

Features	Selection	Insertion	Shell
Best case	$O(n^2)$	$O(n)$	$O(n \log n)$
Avg. case	$O(n^2)$	$O(n^2)$	$O(n^{4/3}) - O(n^{3/2})$
Worst case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space C.	$O(1)$	$O(1)$	$O(1)$
Stability	Unstable	Stable	Unstable
Efficiency	Poor	Good	Better

Sundaram®
FOR EDUCATIONAL USE

3. What is external & internal sorting, in place, stable features of sorting algorithms?

Ans. Internal sorting: Entire data fits in main memory
Eg. insertion

External sorting: Data is too large to fit in ram
Eg. external merge sort.

Stable sorting: Preserves relative order of equal elements.
Eg. insertion sort

Unstable sorting: May change the order of equal elements
Eg. selection sort.

Conclusion:

Thus, implemented different searching and sorting methods on the student database. This system is able to perform sorting under different cases.

Sundaram®
FOR EDUCATIONAL USE

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

// Structure definition according to your pseudocode
struct Student {
    char name[50];
    long long prn;
    int rollNo;
    char mobile[15];
    char panel[10];
};

void inputStudents(struct Student s[], int n);
void displayStudents(struct Student s[], int n);
int linearSearch(struct Student s[], int n, int target);
int binarySearch(struct Student s[], int n, int target);
void insertionSort(struct Student s[], int n);
void selectionSort(struct Student s[], int n);
void shellSort(struct Student s[], int n);

int main() {
    struct Student s[MAX];
    int n = 0, choice, target, pos, isSorted = 0;

    printf("Enter initial number of students: ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAX) {
        printf("Invalid number of students.\n");
        return 1;
    }

    inputStudents(s, n);
```

```
    while (1) {
        printf("\n========================================\n");
        printf("        STUDENT DATABASE MENU          \n");
        printf("========================================\n");
        printf("1. Display Student Records\n");
        printf("2. Search Student by Roll No (Linear Search)\n");
        printf("3. Search Student by Roll No (Binary Search)\n");
        printf("4. Sort Database by Roll No (Insertion Sort)\n");
        printf("5. Sort Database by Roll No (Selection Sort)\n");
        printf("6. Sort Database by Roll No (Shell Sort)\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                displayStudents(s, n);
                break;

            case 2:
                printf("Enter Roll Number to search: ");
                scanf("%d", &target);
                pos = linearSearch(s, n, target);
                if (pos != -1) {
                    printf("\n[FOUND] Record found at index %d:\n", pos);
                    printf("Roll No: %d | Name: %s | PRN: %lld | Mobile: %s | Panel:
%s\n",
                           s[pos].rollNo, s[pos].name, s[pos].prn, s[pos].mobile,
s[pos].panel);
                } else {
                    printf("\n[NOT FOUND] Student with Roll No %d does not exist.
\n", target);
                }
                break;
```

```
        case 3:
            if (!isSorted) {
                printf("\n[NOTICE] Database is unsorted. Sorting
automatically using Shell Sort before Binary Search...\n");
                shellSort(s, n);
                isSorted = 1;
            }
            printf("Enter Roll Number to search: ");
            scanf("%d", &target);
            pos = binarySearch(s, n, target);
            if (pos != -1) {
                printf("\n[FOUND] Record found at index %d:\n", pos);
                printf("Roll No: %d | Name: %s | PRN: %lld | Mobile: %s | Panel:
%s\n",
                       s[pos].rollNo, s[pos].name, s[pos].prn, s[pos].mobile,
s[pos].panel);
            } else {
                printf("\n[NOT FOUND] Student with Roll No %d does not exist.
\n", target);
            }
            break;

        case 4:
            insertionSort(s, n);
            isSorted = 1;
            printf("\n[SUCCESS] Database sorted using Insertion Sort.\n");
            displayStudents(s, n);
            break;

        case 5:
            selectionSort(s, n);
            isSorted = 1;
            printf("\n[SUCCESS] Database sorted using Selection Sort.\n");
            displayStudents(s, n);
            break;
```

```
            case 6:
                shellSort(s, n);
                isSorted = 1;
                printf("\n[SUCCESS] Database sorted using Shell Sort.\n");
                displayStudents(s, n);
                break;

            case 7:
                printf("\nExiting Student Database System. Goodbye!\n");
                exit(0);

            default:
                printf("\nInvalid choice! Please select a valid option.\n");
        }
    }
    return 0;
}

void inputStudents(struct Student s[], int n) {
    for (int i = 0; i < n; i++) {
        printf("\n--- Enter details for Student %d ---\n", i + 1);
        printf("Roll Number: ");
        scanf("%d", &s[i].rollNo);
        printf("Name: ");
        scanf("%49s", s[i].name);
        printf("PRN: ");
        scanf("%lld", &s[i].prn);
        printf("Mobile: ");
        scanf("%14s", s[i].mobile);
        printf("Panel: ");
        scanf("%9s", s[i].panel);
    }
}

void displayStudents(struct Student s[], int n) {
    if (n == 0) {
```

```
        printf("\nDatabase is empty.\n");
        return;
    }
    printf("\n-------------------------------------------------------------------------\n");
    printf("%-10s %-20s %-16s %-15s %-10s\n", "Roll No", "Name", "PRN",
"Mobile", "Panel");
    printf("-------------------------------------------------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-10d %-20s %-16lld %-15s %-10s\n",
               s[i].rollNo, s[i].name, s[i].prn, s[i].mobile, s[i].panel);
    }
    printf("-------------------------------------------------------------------------\n");
}

int linearSearch(struct Student s[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (s[i].rollNo == target) {
            return i;
        }
    }
    return -1;
}

int binarySearch(struct Student s[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (s[mid].rollNo == target) {
            return mid;
        } else if (target < s[mid].rollNo) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return -1;
```

```
}

void insertionSort(struct Student s[], int n) {
    for (int i = 1; i < n; i++) {
        struct Student key = s[i];
        int j = i - 1;
        while (j >= 0 && s[j].rollNo > key.rollNo) {
            s[j + 1] = s[j];
            j--;
        }
        s[j + 1] = key;
    }
}

void selectionSort(struct Student s[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minPos = i;
        for (int j = i + 1; j < n; j++) {
            if (s[j].rollNo < s[minPos].rollNo) {
                minPos = j;
            }
        }
        if (minPos != i) {
            struct Student temp = s[i];
            s[i] = s[minPos];
            s[minPos] = temp;
        }
    }
}

void shellSort(struct Student s[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            struct Student temp = s[i];
            int j = i;
            while (j >= gap && s[j - gap].rollNo > temp.rollNo) {
```

```
                s[j] = s[j - gap];
                j -= gap;
            }
            s[j] = temp;
        }
    }
}
```

Input:

Data	Switch choices
101 Rohan 1262250101 1234567890 A	1
102 Anjili 1262250102 1234567891 A	2 99
103 Rahul	3 12

1262250103 1234567892 B	
104 Aman 1262250104 1234567893 D	4
105 Tina 1262250105 1234567894 C	5
	6
	7

Output:

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe
Enter initial number of students:5


--- Enter details for Student 1 ---
Roll Number:101

Name:Rohan

PRN:1262250101

Mobile:1234567890

Panel:A


--- Enter details for Student 2 ---
Roll Number:102

Name:Anjili

PRN:1262250102

Mobile:1234567891

Panel:A
```

```

--- Enter details for Student 3 ---
Roll Number:103

Name:Rahul

PRN:1262250103

Mobile:1234567892

Panel:B


--- Enter details for Student 4 ---
Roll Number:104

Name:Aman

PRN:1262250103

Mobile:1234567893

Panel:D
```

```
--- Enter details for Student 5 ---
Roll Number:105

Name:Tina

PRN:1262250105

Mobile:1234567894

Panel:C


========================================
         STUDENT DATABASE MENU
========================================
1. Display Student Records
2. Search Student by Roll No (Linear Search)
3. Search Student by Roll No (Binary Search)
4. Sort Database by Roll No (Insertion Sort)
5. Sort Database by Roll No (Selection Sort)
6. Sort Database by Roll No (Shell Sort)
7. Exit
Enter your choice:1
```

```
--------------------------------------------------------------------------------
Roll No    Name                PRN             Mobile          Panel
--------------------------------------------------------------------------------
101        Rohan               1262250101      1234567890      A
102        Anjili              1262250102      1234567891      A
103        Rahul               1262250103      1234567892      B
104        Aman                1262250103      1234567893      D
105        Tina                1262250105      1234567894      C
--------------------------------------------------------------------------------

========================================
         STUDENT DATABASE MENU
========================================
1. Display Student Records
2. Search Student by Roll No (Linear Search)
3. Search Student by Roll No (Binary Search)
4. Sort Database by Roll No (Insertion Sort)
5. Sort Database by Roll No (Selection Sort)
6. Sort Database by Roll No (Shell Sort)
7. Exit
Enter your choice:2

Enter Roll Number to search:99


[NOT FOUND] Student with Roll No 99 does not exist.
```

```
========================================
         STUDENT DATABASE MENU
========================================
1. Display Student Records
2. Search Student by Roll No (Linear Search)
3. Search Student by Roll No (Binary Search)
4. Sort Database by Roll No (Insertion Sort)
5. Sort Database by Roll No (Selection Sort)
6. Sort Database by Roll No (Shell Sort)
7. Exit
Enter your choice:3


[NOTICE] Database is unsorted. Sorting automatically using Shell Sort before Binary Search...
Enter Roll Number to search:12


[NOT FOUND] Student with Roll No 12 does not exist.

========================================
         STUDENT DATABASE MENU
========================================
1. Display Student Records
2. Search Student by Roll No (Linear Search)
3. Search Student by Roll No (Binary Search)
4. Sort Database by Roll No (Insertion Sort)
5. Sort Database by Roll No (Selection Sort)
6. Sort Database by Roll No (Shell Sort)
7. Exit
```

```
Enter your choice:4


[SUCCESS] Database sorted using Insertion Sort.

-------------------------------------------------------------------------------
Roll No    Name                PRN             Mobile          Panel
-------------------------------------------------------------------------------
101        Rohan               1262250101      1234567890      A
102        Anjili              1262250102      1234567891      A
103        Rahul               1262250103      1234567892      B
104        Aman                1262250103      1234567893      D
105        Tina                1262250105      1234567894      C
-------------------------------------------------------------------------------

======================================
        STUDENT DATABASE MENU
======================================
1. Display Student Records
2. Search Student by Roll No (Linear Search)
3. Search Student by Roll No (Binary Search)
4. Sort Database by Roll No (Insertion Sort)
5. Sort Database by Roll No (Selection Sort)
6. Sort Database by Roll No (Shell Sort)
7. Exit
Enter your choice:5
```

```
[SUCCESS] Database sorted using Selection Sort.

-------------------------------------------------------------------------------
Roll No    Name                PRN             Mobile          Panel
-------------------------------------------------------------------------------
101        Rohan               1262250101      1234567890      A
102        Anjili              1262250102      1234567891      A
103        Rahul               1262250103      1234567892      B
104        Aman                1262250103      1234567893      D
105        Tina                1262250105      1234567894      C
-------------------------------------------------------------------------------

======================================
        STUDENT DATABASE MENU
======================================
1. Display Student Records
2. Search Student by Roll No (Linear Search)
3. Search Student by Roll No (Binary Search)
4. Sort Database by Roll No (Insertion Sort)
5. Sort Database by Roll No (Selection Sort)
6. Sort Database by Roll No (Shell Sort)
7. Exit
Enter your choice:6


[SUCCESS] Database sorted using Shell Sort.
```

```
[SUCCESS] Database sorted using Shell Sort.

-------------------------------------------------------------------------------
Roll No    Name                PRN             Mobile          Panel
-------------------------------------------------------------------------------
101        Rohan               1262250101      1234567890      A
102        Anjili              1262250102      1234567891      A
103        Rahul               1262250103      1234567892      B
104        Aman                1262250103      1234567893      D
105        Tina                1262250105      1234567894      C
-------------------------------------------------------------------------------

======================================
        STUDENT DATABASE MENU
======================================
1. Display Student Records
2. Search Student by Roll No (Linear Search)
3. Search Student by Roll No (Binary Search)
4. Sort Database by Roll No (Insertion Sort)
5. Sort Database by Roll No (Selection Sort)
6. Sort Database by Roll No (Shell Sort)
7. Exit
Enter your choice:7


Exiting Student Database System. Goodbye!

Process finished with exit code 0
```